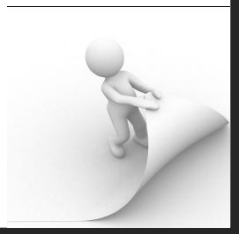
C++11新特性之一：保持稳定性和兼容性

张锦成 2015/10/12

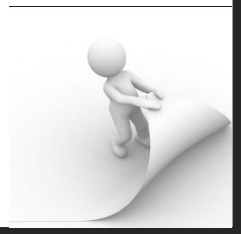


Outline

- C++11标准的诞生
  - 保持与C99兼容
    - 静态断言static_assert
      - 就地初始化成员变量与初始化列表

1

C++11标准的诞生



C++11诞生

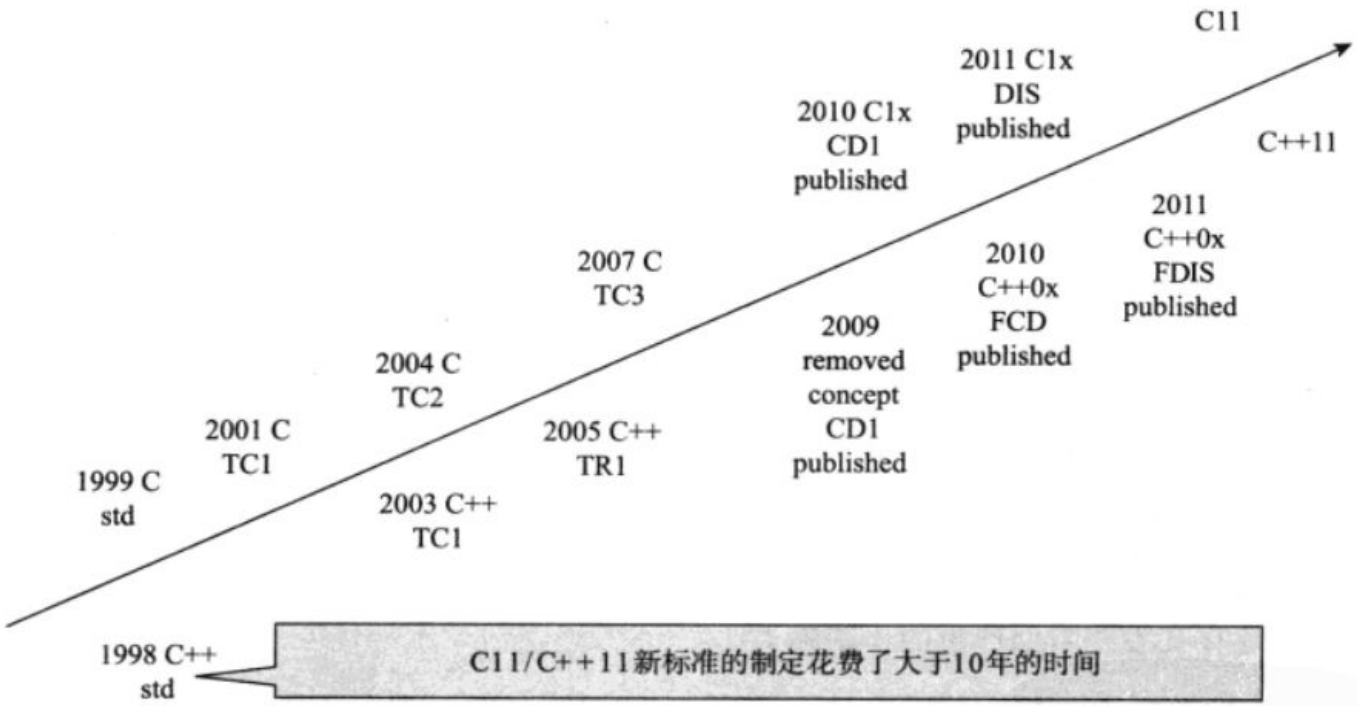
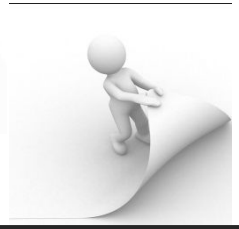
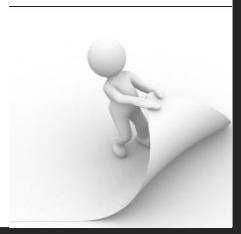


图 1-1　WG21 和 WG14 制定新语言标准的工作进程



2

保持与C99兼容

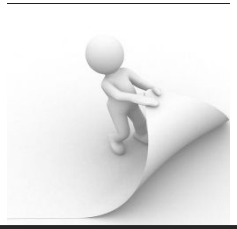


兼容C99

兼容C99中新添加的预定义宏

表 2-1　C++11 中与 C99 兼容的宏

宏 名 称	功 能 描 述
__STDC_HOSTED__	如果编译器的目标系统环境中包含完整的标准 C 库，那么这个宏就定义为 1，否则宏的值为 0
__STDC__	C 编译器通常用这个宏的值来表示编译器的实现是否和 C 标准一致。C++11 标准中这个宏是否定义以及定成什么值由编译器来决定
__STDC_VERSION__	C 编译器通常用这个宏来表示所支持的 C 标准的版本，比如 1999mmL。C++11 标准中这个宏是否定义以及定成什么值将由编译器来决定
__STDC_ISO_10646__	这个宏通常定义为一个 yyyymmL 格式的整数常量，例如 199712L，用来表示 C++ 编译环境符合某个版本的 ISO/IEC 10646 标准



兼容C99

__func__预定义标识符

```
#include <iostream>

std::string hello() {
    return __func__;
}

int main()
{
    std::cout << hello() << std::endl;
    return 0;
}
```

```
struct Test{
    Test():name(__func__) {}
    std::string name;
};
```

轻量级代码调试

兼容C99

- _Pragma操作符, _Pragma(字符串字面值)

```
#pragma once

_Pragma("once")
```

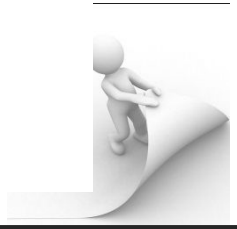
兼容C99

- 变长参数的宏定义以及__VA_ARGS__

变长参数的宏定义：宏定义中参数列表的最后一个参数为省略号

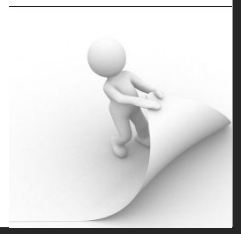
__VA_ARGS__则可以在宏定义的实现部分替换省略号所表示的字符串

```
#include <iostream>

#define LOG(...) {\
    fprintf(stderr, "%s: Line %d:\t", __FILE__, __LINE__);\
    fprintf(stderr, __VA_ARGS__);\
    fprintf(stderr, "\n");\
}

int main()
{
    int x = 3;
    LOG("x = %d", x);
    return 0;
}
```



3

宏__cplusplus



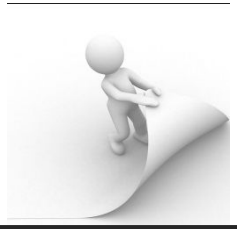
__cplusplus

__cplusplus被定义为一个整型值
通过这个宏可以检测编译程序时所用的C++标准版本

C++98/03中被预定义为199711L

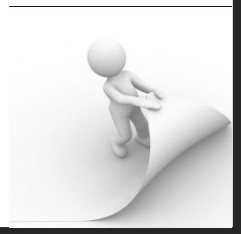
C++11中被预定义为201103L

```
#if __cplusplus < 201103L
    #error "should use C++11 implementation"
#endif
```



4

静态断言static_assert



static_asssert

- 静态断言：编译阶段进行断言

assert: 程序运行阶段起作用

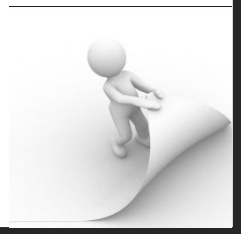
static_asssert

静态断言：编译阶段进行断言，提早发现程序错误

```
template <typename T, typename U>
int bit_copy(T & a, U & b) {
    static_assert(sizeof(a)==sizeof(b), "size does not match!");
    memcpy(&a, &b, sizeof(b));
}

int main() {
    int a = 6;
    double b;
    bit_copy(b, a);
    std::cout << b << std::endl;
    return 0;
}
```

5

就地初始化成员变量与初始化列表



成员变量初始化

回顾：C++98是如何初始化类的成员变量的？

普通，static, const, const static

```
class Test{
    public:
        Test(int tmp): i(tmp), k(tmp) {
            std::cout << "i=" << i << " j=" << " k=" << k << " m=" << m << std::endl;
        }
    private:
        int i;
        static int j;
        const int k;
        const static int m = 10;
};

int Test::j = 5;
```

成员变量初始化

C++11可以就地初始化类的成员变量

普通，static, const, const static

```
class Test{
    public:
        Test(){
            std::cout << "i=" << i << " j=" << j <<  " k=" << k << " m=" << m << std::endl;
        }
    private:
        int i = 10;
        static int j;
        const int k = {10};
        const static int m {10};
};

int Test::j = 5;
```

初始化列表{}

- C++11: 初始化列表{}简化了模板泛型编程

```
int a[] = {1, 3, 5};

int b[]{2, 4, 6};

std::vector<int> vec{1, 3, 5};

std::map<int, std::string> m = {{1, "a"}, {2, "b"}, {3, "c"}};
```

初始化列表{}

- C++11: 初始化列表{}简化了模板泛型编程

```
vector<int> func() {
    return {1, 3};
}
```

初始化列表{}

- 除了内置类型外，初始化列表也可用于自定义类型

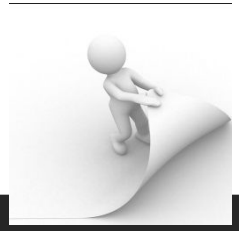
```
enum Gender {
    boy,
    girl
};
class People{
    public:
        People(std::initializer_list<std::pair<std::string, Gender>> l) {
            for(auto i=l.begin(); i!=l.end(); ++i) {
                data.push_back(*i);
            }
        }
    private:
        std::vector<std::pair<std::string, Gender>> data;
};
People people = {{"GarField", boy}, {"HelloKitty", girl}};
```

References

1. 《深入理解C++11: C++11新特性解析与应用》第2章和第3章

2. C++11新特性: http://www.cocos.com/doc/tutorial/show?id=1120

3. http://cpprocks.com/c11-compiler-support-shootout-visual-studio-gcc-clang-intel/



THANK
YOU!